

BLOG / COMMUNITY

JAMIE LYNCH

SEPTEMBER 7, 2017 / 4 MIN

The Worst Computer Bugs in History: The Ariane 5 Disaster

The Worst Computer Bugs in History is a mini series to commemorate [the discovery of the first computer bug](#) seventy years ago. Although these stories are more extreme than most software bugs engineers will encounter during their careers, they are worth studying for the insights they can offer into software development and deployment. These computer bugs left a significant impact on the people who experienced them, and we hope they'll offer valuable lessons we can all apply to our own work and projects. Read about other computer bugs in the series: [The Mars Climate Orbiter, losing \\$460 million in 45 minutes](#), and [Therac-25](#).

Ariane 5

On June 4th, 1996, the very first Ariane 5 rocket ignited its engines and began speeding away from the coast of French Guiana. 37 seconds later, the rocket [flipped 90 degrees](#) in the wrong direction, and less than two seconds later, aerodynamic forces ripped the boosters apart from the main stage at a height of 4km. This caused the self-destruct mechanism to trigger, and the spacecraft was consumed in a gigantic fireball of liquid hydrogen.

The disastrous launch cost approximately \$370m, led to a public inquiry, and through the destruction of the rocket's payload, delayed scientific research into workings of the Earth's magnetosphere for almost 4 years. The Ariane 5 launch is widely acknowledged as one of the most expensive software failures in history.

What went wrong?

The fault was quickly identified as a software bug in the rocket's Inertial Reference System. The rocket used this system to determine whether it was pointing up or down, which is formally known



as the horizontal bias, or informally as a BH value. This value was represented by a 64-bit floating variable, which was perfectly adequate.

However, problems began to occur when the software attempted to stuff this 64-bit variable, which can represent billions of potential values, into a 16-bit integer, which can only represent 65,535 potential values. For the first few seconds of flight, the rocket's acceleration was low, so the conversion between these two values was successful. However, as the rocket's velocity increased, the 64-bit variable exceeded 65k, and became too large to fit in a 16-bit variable. It was at this point that the processor encountered an operand error, and populated the BH variable with a diagnostic value.

Not enough space to reach space

In layman's terms, this can be thought of as attempting to fit 10 million liters of ice cream into a camping fridge on a hot summer's day. It'll be fine for the first few tubs, but after a certain threshold, you'll be unable to fit anything else in, the fridge door will be stuck wide open, and everything will start melting really, *really* fast.

The backup Inertial Reference System also failed due to the same error condition, meaning that at T+37 the BH variable contained a diagnostic value from the processor, intended for debugging purposes only. This was mistakenly interpreted as actual flight data, and caused the engines to immediately over-correct by thrusting in the wrong direction, resulting in the destruction of the rocket seconds later.

It worked on the last device

Several factors make this failure particularly galling. Firstly, the BH value wasn't even required after launch, and had simply been left in the codebase from the rocket's predecessor, the Ariane 4, which did require this value for post-launch alignment. Secondly, code which would have caught and handled these conversion errors had been disabled for the BH value, due to performance constraints on the Ariane 4 hardware which did not apply to Ariane 5.

A final contributing factor was a change in user requirements – specifically in the rocket's flight plan. The Ariane 5 launched with a much steeper trajectory than the Ariane 4, which resulted in greater vertical velocity. As the rocket sped to space faster, there was a higher certainty that the BH value would encounter the conversion error.

Ultimately, the European Space Agency assembled a team to recover logs from the two Inertial Reference Systems, which were spread over a debris field of approximately 12 square kilometers. Their work was impeded by treacherous marshland terrain, hazardous chemicals dispersed from the rocket, and immense public scrutiny from the media, all because of a single type casting error.

Our series on the Worst Software Bugs in History is in honor of [Bug Day 2017](#). Seventy years ago, Grace Hopper discovered the first computer bug — a moth was stuck between relays in the Harvard Mark II computer she was working on. The notion of bugs was described in other fields previously, but the moth discovery was the first use of the term "debugging" in the field of computers.

[BugSnag](#) automatically monitors your applications for harmful errors and alerts you to them, giving you visibility into the stability of your software. You can think of us as mission control for software quality.

Sources:

https://en.wikipedia.org/wiki/Ariane_5#Notable_launches
[https://en.wikipedia.org/wiki/Cluster_\(spacecraft\)#Launch_failure](https://en.wikipedia.org/wiki/Cluster_(spacecraft)#Launch_failure)
<https://web.archive.org/web/20000815230639/>
<http://www.esrin.esa.it/htdocs/tidc/Press/Press96/ariane5rep.html>